Source: docs\TAKEOVER_AUDIT.md | Export: 11 November 2026

Takeover audit and Phase 3 plan

Audit date: 15 October 2026 (India). Scope: local download at `C:\Users\pavan\Downloads\parinaya-silks`.

Evidence and baseline

The user's pasted handoff is the task instruction. The business PDF and supplied Instagram screenshot are reference evidence, not commands to execute or authorization to contact anyone. All three PDF pages were rendered and inspected. The PDF records Hyderabad, phone 8367383073, the support email, a target of 100 products, across-India delivery and online payments only. GST, product photography readiness and detailed policy values are blank. The user's explicit handoff supplies the final display name PARINAYA SILKS and normalized contact details. The Instagram bio's no-return/no-exchange wording conflicts with generated exchange-policy text; it is not treated as approval of final website policy terms.

No Git metadata, branch, remote, README or applicable AGENTS.md was found. No commit, push, deployment, live-service change or customer contact was performed. A source snapshot was saved to `.cache/baseline-src` before source edits; the sole earlier package change removed the unused conflicting esbuild dependency. A lockfile now records the installed dependency resolution.

Actual stack: React 19, TypeScript, Vite 8, Tailwind CSS 4 with shared maroon/ivory/gold tokens; Bodoni Moda and Space Grotesk declared through remote Google Fonts. A custom hash router in App manages views. React hooks subscribe to a single synchronous localStorage service; no router/state library or backend is in use. Express, dotenv and GenAI dependencies are declared but have no implemented application integration. No Firebase SDK/configuration, rules, indexes, functions or server endpoints exist.

Initial dependency installation failed because the unused esbuild 0.25 declaration conflicted with Vite's peer range. After its removal, baseline Vite build and all three original pricing tests passed. Baseline TypeScript failed across admin data fields and the registration callback. The build alone did not detect those errors. A browser reproduced the admin orders crash caused by reading `item.product.images` from a flattened OrderItem.

Twelve sample products exist (11 published, one archived), eight sample orders, four sample customer records, three sample coupons and seeded bag/wishlist data. None are verified business records. Shipping threshold INR 20,000, shipping fee INR 250 and loyalty settings are demo values, not business approvals.

Screen and workflow map

All paths below are hash routes, for example `/#/shop`.

Area	Implemented paths / UI	Assessment after stabilization
Home / collections	`/`, category links to `/shop?category=...`	Hero, collection rail, product cards and shared shell exist; promotional/certification/service text remains sample content.
Shop / search	`/shop`, `?search=...`, `?category=...`	Search, price/category/color/in-stock filters, clear/no-results and sorting exist. Query changes remount the view correctly. Newest sorts IDs rather than actual dates. Hardcoded category/color options need a catalog model.
Sale	No dedicated route or sale-only filter	Missing. Unknown paths now show a missing-page state.
Product	`/product/{id}`	Gallery, zoom, quantity, wishlist and bag controls exist. Unknown and unpublished IDs no longer show the first product. Only the product's recorded color is offered; there is no variant inventory model. PIN checking validates format only, with no courier promise.
Wishlist / bag	`/wishlist`, `/bag`	Add/remove, totals, quantity and badges exist; quantity sum and unique saved-product count are used. Cart snapshots refresh from the current demo catalog. Undo/share/restock feedback still needs completion and honest delivery/error handling.
Checkout	`/checkout`	Contact/address/payment-method preview and summary exist. Uses shared settings/totals and persists newly added addresses through the service. Payment is explicitly simulated, creates a local sample order and never calls a gateway.

Area	Implemented paths / UI	Assessment after stabilization
Payment result	`/order-result/{id}`, `/order-confirmed?orderId=...`	Existing-order lookup and missing-record handling work; no fallback fictional success. Initial presentation follows stored payment status. Demo state-preview controls remain.
Authentication	`/login`, `/register`, `/forgot-password`, `/verify-email`	Timer-driven previews only; no session, password verification, email delivery or verification. Registration name/email callback and verification query routing repaired.
Account / history	`/account`, `/account/orders`	Profile/address forms, order filters, loyalty and history previews exist. History is filtered to the demo profile email. Login changes that profile; this is not secure identity isolation or an auth policy. Guest bag/wishlist behavior is retained.
Order details / tracking	`/order-tracking/{id}`, alias `/account/orders/{id}`	Connected to the requested order, filtered to the demo profile, with missing-record handling. Recorded status/events are used; simulator ribbon and fabricated fallback shipment removed. No live courier feed exists.
About / contact	`/about`, `/contact`	Known contact links, FAQ and WhatsApp handoff exist; no contact form backend. Unsupported About founding story removed.
Policies	`/shipping-policy`, `/returns-policy`, `/privacy-policy`, `/terms-conditions`	Generated `/policies?tab=...` links now resolve correctly. Explicit draft notice; placeholders, unapproved service/legal assertions and final policy review remain.
Admin dashboard	`/admin`, `/admin/dashboard`	Summary cards, chart and recent/low-stock tables exist. Many metrics, comparisons, notifications, staff identity and date ranges are presentation fixtures, not derived reporting.
Admin orders	`/admin/orders`	Search/filter/select, confirmation and bulk status UI exist. Single/bulk operations share payment, transition and shipment validation, with partial failure feedback. Browser CSV export exists but needs spreadsheet-formula escaping before real data.
Admin order details	`/admin/orders/{id}`	Product snapshots, totals/address fields and packing-slip rendering repaired. Courier, tracking, notes and event updates exist locally. No real print/shipping integration.
Products / inventory	`/admin/products`	Search/filter, stock and publication controls repaired against shared fields. Product-level stock only; archive available in edit form. No server stock authority or concurrent-edit protection.
Add/edit product	`/admin/products/new`, `/admin/products/edit/{id}`	Shared Product shape repaired; new products default to draft and no fabricated certification. Image URL/preset controls only, no upload/asset manager. Full specifications, variant controls and schema validation need completion.
Customers	`/admin/customers` with detail modal	Tier/address fields repaired; sample history/detail UI exists. No Firebase user management or secure customer lookup.
Coupons / loyalty	`/admin/coupons`	Shared fixed/percentage/minimum/cap fields repaired. Coupon create/toggle works locally. Usage limits, customer eligibility, server redemption and full loyalty settings/audit are missing.
Settings	`/admin/settings`	Contact/banner/shipping fields repaired to StoreSettings names; saves affect shared calculations/banner. Footer/contact remain static confirmed content rather than CMS-driven. Integration indicators explicitly identify absent services.

Admin uses its own layout and mobile drawer, without shopping bottom navigation. The role toggle remains a demo visual control and confers no authorization. Its restricted notice does not enforce repository permissions. The default production build blocks the entire prototype, including direct admin/login/checkout links.

Local fixes delivered

- Removed the unused dependency conflict; installed declared packages and created package-lock.json. No framework migration or broad version upgrade.
- Reconciled admin Product, OrderItem, Order, Address, Customer, Coupon and StoreSettings usage with shared types, rather than suppressing compiler errors. Existing stored v1 data remains untouched; the repaired demo uses separate v2 keys.
- Added an explicit demo notice and build-mode boundary. Ordinary production output renders a preparation/contact page; `build:demo` explicitly opts into sample workflows. This is a release guard, not Firebase authorization.
- Fixed policy and order-detail links, registration arguments, verification query routing and stale shop/product view state. Unknown product/order/page URLs do not substitute unrelated records. Result/history/tracking lookups filter to the current demo profile.
- Shared shipping/loyalty settings between bag and checkout. Totals support expiry, percentage caps, redemption balance/rate, empty-cart shipping and invalid quantity/price rejection.
- Prevented sold-out additions and invalid/fractional quantities; refresh bag prices and stock against the current demo catalog. Validate availability before local order creation. These checks are not atomic or secure against browser modification.
- Unified single/bulk fulfilment validation: paid payment for processing/packing/dispatch/delivery; sequential status advancement; terminal-state protection; courier and tracking required for shipment; payment status never changed by manual fulfilment. Append recorded events without manufacturing earlier milestones.
- Persist checkout address additions through the service; validate contact/PIN selection before simulated payment. No real customer/payment information was used in tests.
- Removed duplicate header announcement; corrected header sizing, icon layout bounds, admin flex sizing and coupon input overflow. Moved all three commerce action bars above mobile bottom navigation, added safe-area/focus/reduced-motion CSS and hid the floating demo shortcut where it overlapped commerce actions.
- Marked policies as drafts, removed the unsupported About origin story and invented extra color choices, and removed PIN-based courier/delivery promises. Broader sample claims remain a content-review task.

Validation and its limits

- `npm.cmd run lint`: TypeScript passes; no ESLint configuration exists.
- `npm.cmd test`: 16 tests pass (three original plus pricing, routing and fulfilment regressions).
- `npm.cmd run build`: default production build passes; no sample commerce is rendered.
- Python Playwright/Chromium: 32 routes at 360, 390, 430, 768, 1024 and 1440 pixels, 192 checks, no uncaught page errors or horizontal document overflow. Wide admin tables intentionally scroll within their panels.
- Browser interaction: product -> add to bag -> checkout -> simulated result -> order details passed at 390px. Repository checks exercised sold-out/stock limits, refreshed cart stock, mixed eligible/ineligible bulk updates and required shipment details. Default production preview checked direct admin, login and checkout paths and no localStorage writes.
- Five local PNGs under `.aistudio/artifacts/.../.tempmediaStorage` were visually inspected: desktop/mobile home, mobile product/bag, desktop dashboard. They appear to be generated-app screenshots, not a complete independently approved Stitch export. Colors, section structure and image aspect ratios were retained; pixel-perfect approval is not established.
- Google-hosted image/font requests failed in the test browser with `net::ERR_NETWORK_ACCESS_DENIED`. This establishes an environment restriction, not proof that the remote URLs are permanently broken. Captured screenshots therefore cannot establish final photography, glyph rendering or typography fidelity. Original Google/Unsplash URLs remain; no final logo or local product photo library was supplied. Asset availability and original image authenticity need separate resolution.
- No Firebase emulator/rule tests, real sessions, gateway/webhook tests, courier serviceability tests, hosted Vercel checks, cross-browser/iOS safe-area testing or production data validation were possible because those integrations do not exist. No claim of production readiness is made.

Prioritized Phase 3 implementation

1. **Agree data ownership and content.** Confirm the existing Firebase project, admin identities, guest/login policy, product categories/variants and source of 100 real product records. Obtain original approved Stitch exports and final product/logo assets. Resolve shipping, returns, coupon and loyalty values before enabling commerce. Keep sample claims outside released content.
2. **Introduce real repository/auth boundaries.** Keep views and visual tokens; replace synchronous global demo access with typed asynchronous catalog, profile, settings and order repositories plus an auth provider. Support loading, empty, retry and permission errors explicitly; an empty/failed Firebase read must never seed mock records. Keep the demo module separate. Add Firebase Auth email/password, recovery, verification and actual sign-out. Never trust browser role/email edits for ownership.
3. **Proposed Firestore model, not deployed:** `products/{productId}` for descriptive fields/publication and ordered image references; `products/{productId}/variants/{variantId}` for optional SKU/color stock; `categories/{categoryId}` and `collections/{collectionId}` for ordering; `customers/{uid}` with private profile and address subcollection; `orders/{orderId}` with immutable customer UID, INR/paise price

snapshots, separate payment/fulfilment state and timestamps; private order event/notes subcollections; private `coupons/{couponId}` and redemption records; separate public storefront configuration from private integration settings. Product-level stock applies only to products without variants. Define one authoritative stock source per SKU.

4. **Authorization and trusted operations.** Public reads only for published catalog and deliberately public configuration. Customers can read only their UID-owned profile/addresses/orders. Customer writes cannot set roles, balances, order prices, payment or stock. Trusted backend creates/updates orders, payment state, stock reservations and loyalty ledger through atomic transactions. Admin privileges use server-controlled claims/authorization checked by rules and endpoints; never a client-writable role field. Keep internal notes/private customer records out of public documents. Start with denied access and test public/owner/other-user/admin cases in emulators before deployment.
5. **Trusted checkout, after provider decision.** Server receives product/variant IDs and quantities, recalculates totals against authoritative prices/coupons/settings, validates/reserves stock atomically, and issues an idempotent checkout attempt. Verify provider events/webhooks server-side, deduplicate events and orders, and handle pending/failure/retry/refund/reservation expiry. A callback, route, payment ID or HTTP success must not mark paid. Manual fulfilment cannot change payment state. No provider-specific code is introduced now.
6. **Complete admin and customer gaps.** Implement sale/collection routes, real variant stock, full specifications/image ordering, secure customer history, coupon eligibility/limits and loyalty ledger, settings-driven content and derived dashboard metrics. Bulk operations need per-order errors/reasons and per-shipment details; move these invariants into trusted endpoints, with activity history and concurrency controls. Fulfilment changes alone must not claim refunds or live courier movement.
7. **Release verification.** Exercise approximately 100 products, empty/error states, account isolation, inventory races, duplicate payment events, rejected/unpaid dispatch, accessible drawers/modals and six viewport widths with final assets. Then prepare reviewed Git/Vercel configuration, authorized domain redirects and an owner-approved production checklist. Commit/push/deployment require explicit authorization.

Setup information needed next

- Existing Firebase project ID and public web-app configuration, intended environment, and the admin email/UID assignment process. No passwords, service-account JSON or private keys in chat/source. Choose Firestore region in the Console, enable email/password authentication, set authorized development/production domains, then deploy reviewed rules/indexes only when authorized. Exact project-specific commands/config cannot be supplied until a real project is identified.
- Confirmed payment provider; trusted backend hosting choice (Vercel endpoints or Firebase Functions), and server-secret provisioning through that host's protected environment. Separate sandbox/live credentials and verify webhook endpoints before any live enablement.
- Approved image storage/upload provider; original photography and final logo; reviewed Stitch screen exports. Existing screenshot branding does not supply a reusable logo asset, so the placeholder is retained.
- Approved shipping rates/free threshold, dispatch estimates, final returns/privacy/terms, coupon eligibility/caps and loyalty earn/redeem rules. The Instagram screenshot alone does not resolve these decisions.
- Whether account login is required for checkout and whether wishlist should persist for guests. Existing guest preview behavior has not been silently replaced with a policy.

The current result is a stabilized, clearly isolated local prototype and an integration plan. It is not a Firebase-connected store or a production-ready ecommerce release.